

論理回路

第12回 順序回路(2)

クロックを持つフリップフロップ回路

授業の目標

- 理解して持ち帰ってほしい事柄
 - SR-フリップフロップに無かった「クロック」の概念を理解
 - クロックに基づいて動作する同期式SR-フリップフロップの動作の理解
 - Dラッチ(Dフリップフロップ)とエッジトリガ式Dフリップフロップの動作の理解
 - SR-フリップフロップ, Dラッチ, エッジトリガ式Dフリップフロップのタイミングチャート
 - 同期式SR-フリップフロップとDラッチを論理回路シミュレータで動作させて実践的に理解
- 授業中の演習
 - 今回も論理回路シミュレータを使った演習を行います

目次

1. SR-フリップフロップ復習
 2. クロックについて
 3. 同期式SR-フリップフロップ
 4. Dラッチ(レベル・センスD-FF)
 5. エッジトリガ式D-フリップフロップ
 6. 【演習】論理回路シミュレータで動作確認
- まとめ

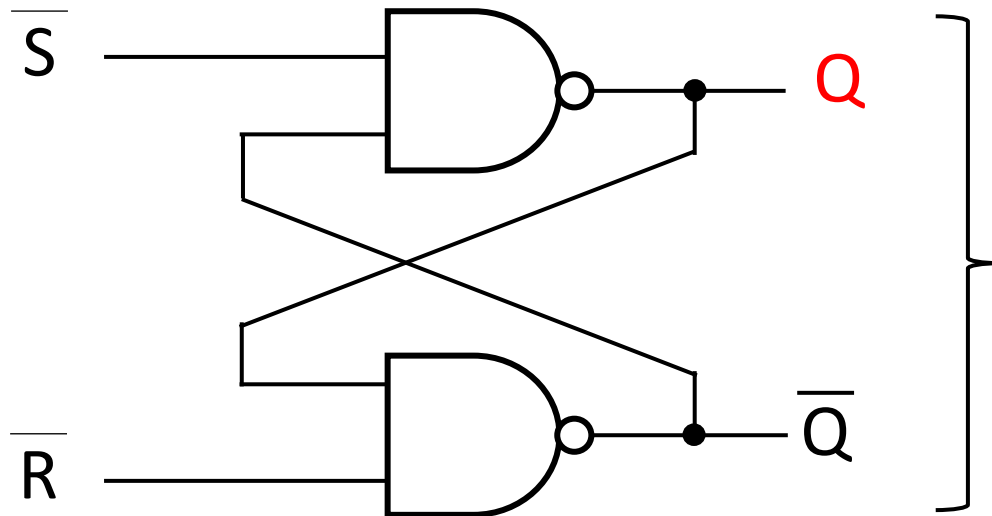
1. SR-フリップフロップ復習

1-1. SR-フリップフロップの回路

- SR-FFの動作(復習)

- 入力に $\sim S$ (セットバー)と $\sim R$ (リセットバー)があり, $\sim S$ に0を入力すると, 出力Qが1になる。 $\sim R$ に0を入力すると, 出力Qが0になる。
- 両方の入力を1にすると安定(固定)される。

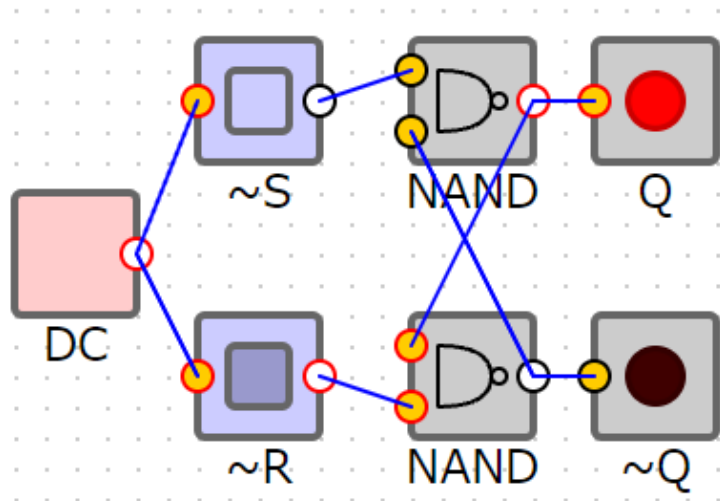
- SR-FFの回路



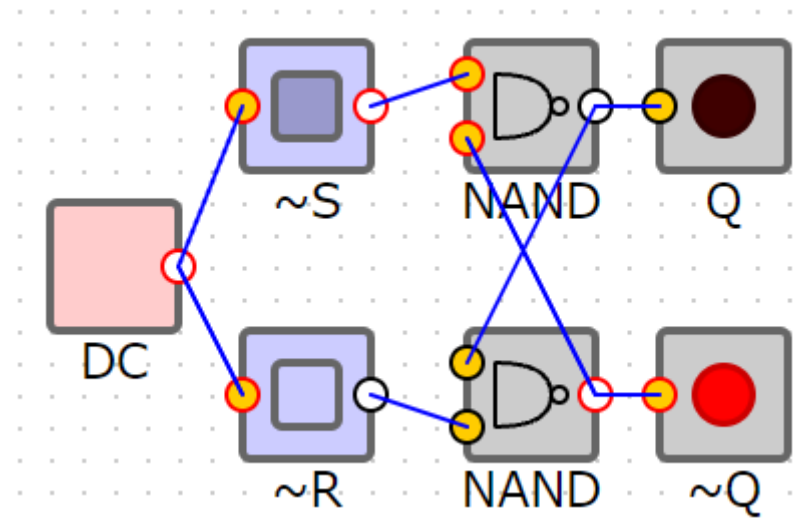
- NANDを2つ組合せ, **一つの出力をもう一つのNANDの入力にする**
- 負論理なので, **入力を0にすると有効になる**

1-2. SR-フリップフロップの動作確認

- 論理回路シミュレータで作成したSR-FFの動作を確認
 - 課題で作成しているはずです
 - これから動作させます。皆さんも一緒に動作確認してください



(1) $\sim S$ を0に設定



(2) $\sim R$ を0に設定

2. クロックについて

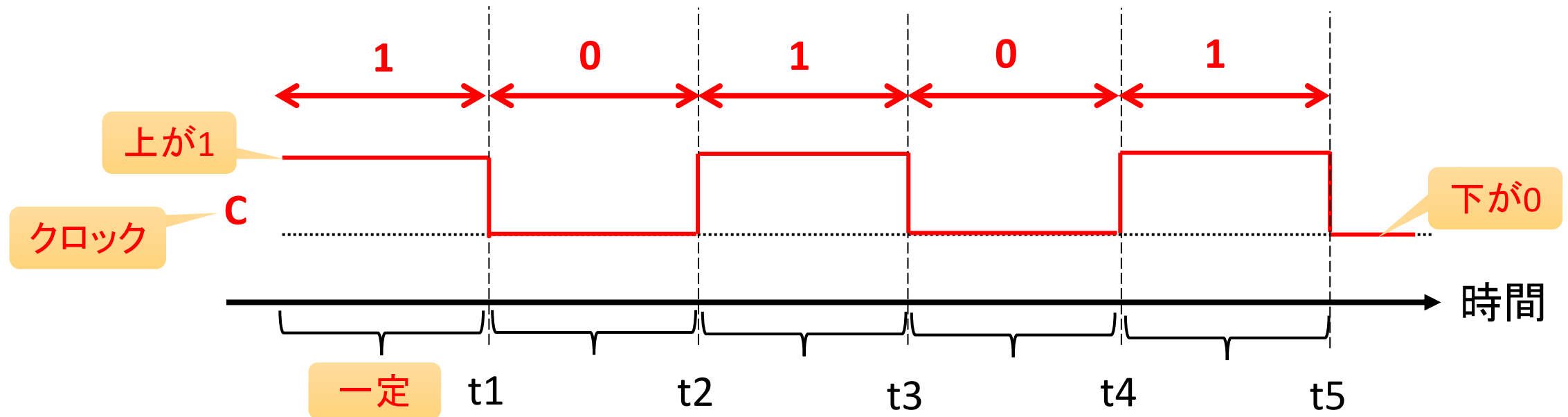
2-1. クロックとは

- クロック(Clock)は, 計算機を構成する機器群の動作を, 同期させる信号のこと
 - 例) 計算機のCPUとメモリの動作を上手く合わせる
- クロックを用いると, 機器群の動作を同期(一致)させられる
- 計算機では, 水晶発振器によって, クロックを生成している
- 水晶発振器の例 (https://www.marutsu.co.jp/contents/shop/marutsu/img/goods/020/575164/575164_2.jpg)



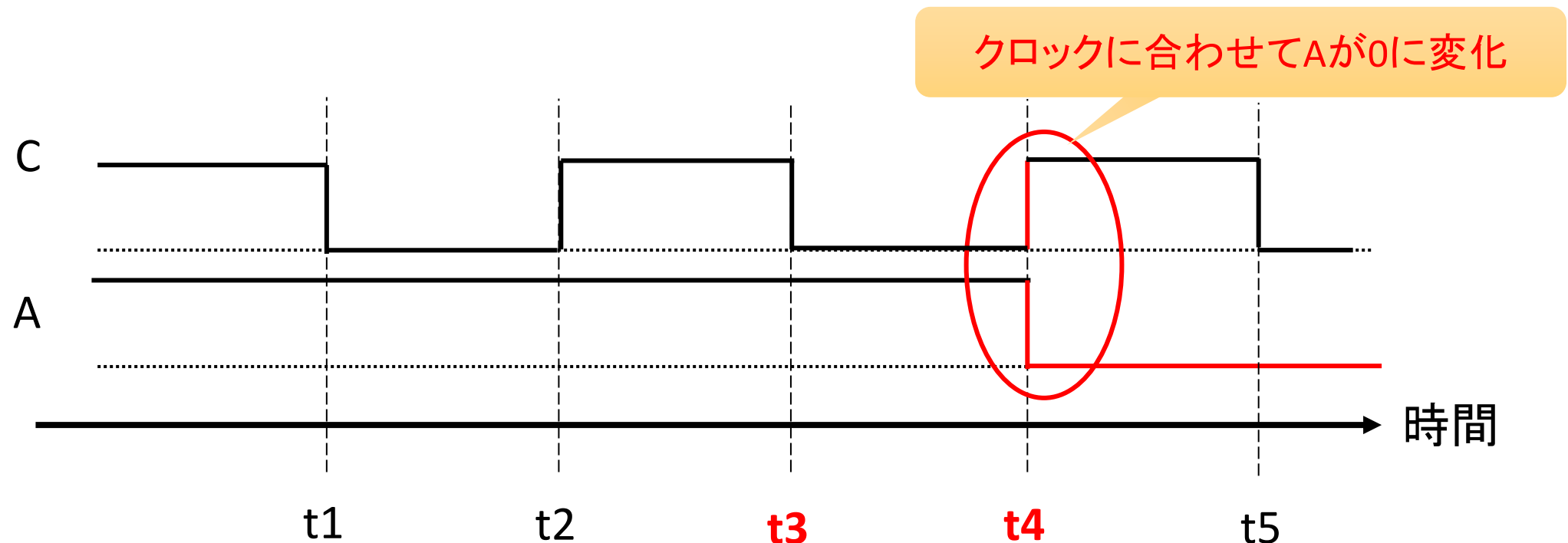
2-2. クロックのタイミングチャート

- クロックは、一定間隔で0と1を繰り返す
- クロックをタイミングチャートに表すと、以下の様になる



2-3. クロックを使った制御

- SR-FFは, $\sim S$ や $\sim R$ が0になった「**任意の**」タイミングで, 出力が変化する
- クロックを使うと, **タイミングを「合わせて」動作**させられる利点がある
- タイミングに合わせて動作する例

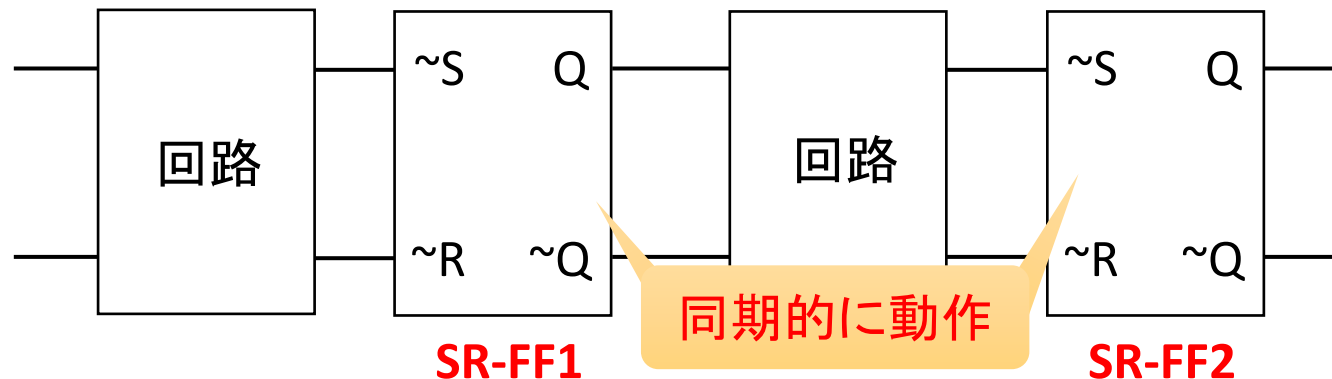


3. 同期式SR-フリップフロップ

3-1. SR-フリップフロップの難点

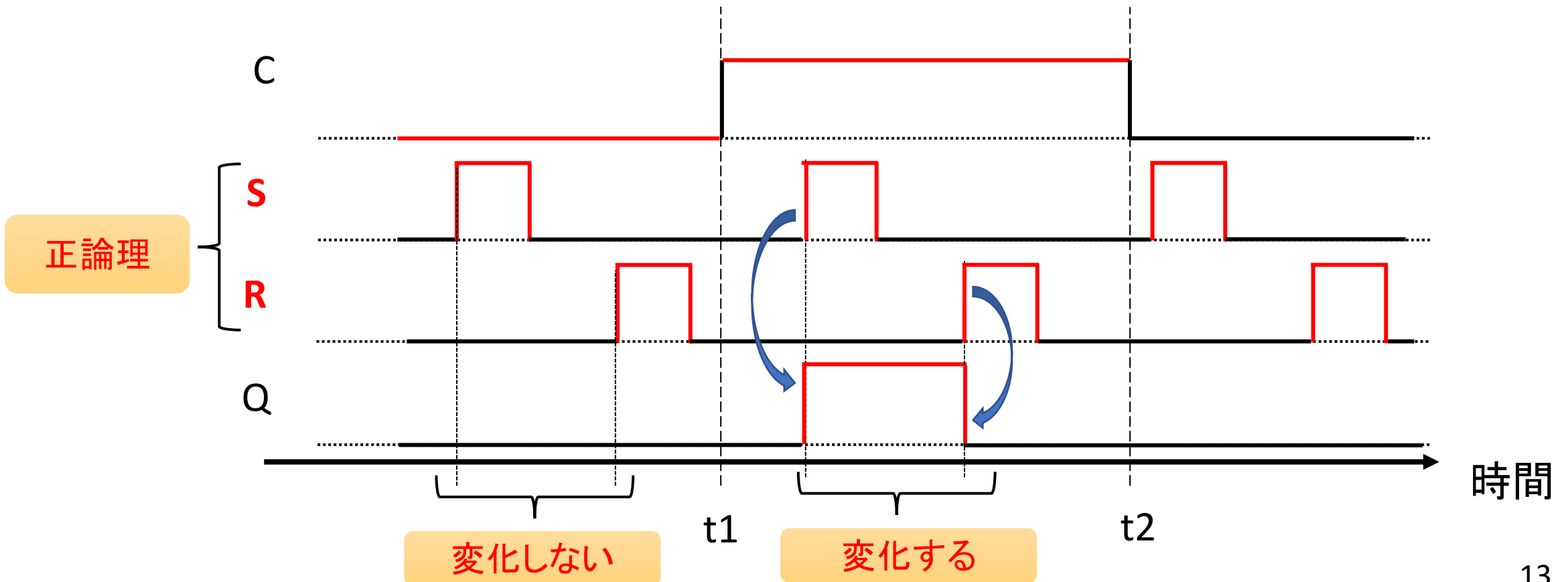
- SR-FFは、 $\sim S$ や $\sim R$ に0を設定すると、出力Qがすぐに1や0に設定される
- SR-FFを複数使ったり、他の回路と組み合わせて使う場合、同期を取ることができず、回路の動作結果が不定になる場合がある
 - 下手すると、**禁止入力**($\sim S=\sim R=0$)になってしまうかも

全体で同期をとり、論理回路を確実に動作させたい



3-2. クロックに合わせたSet/Reset動作

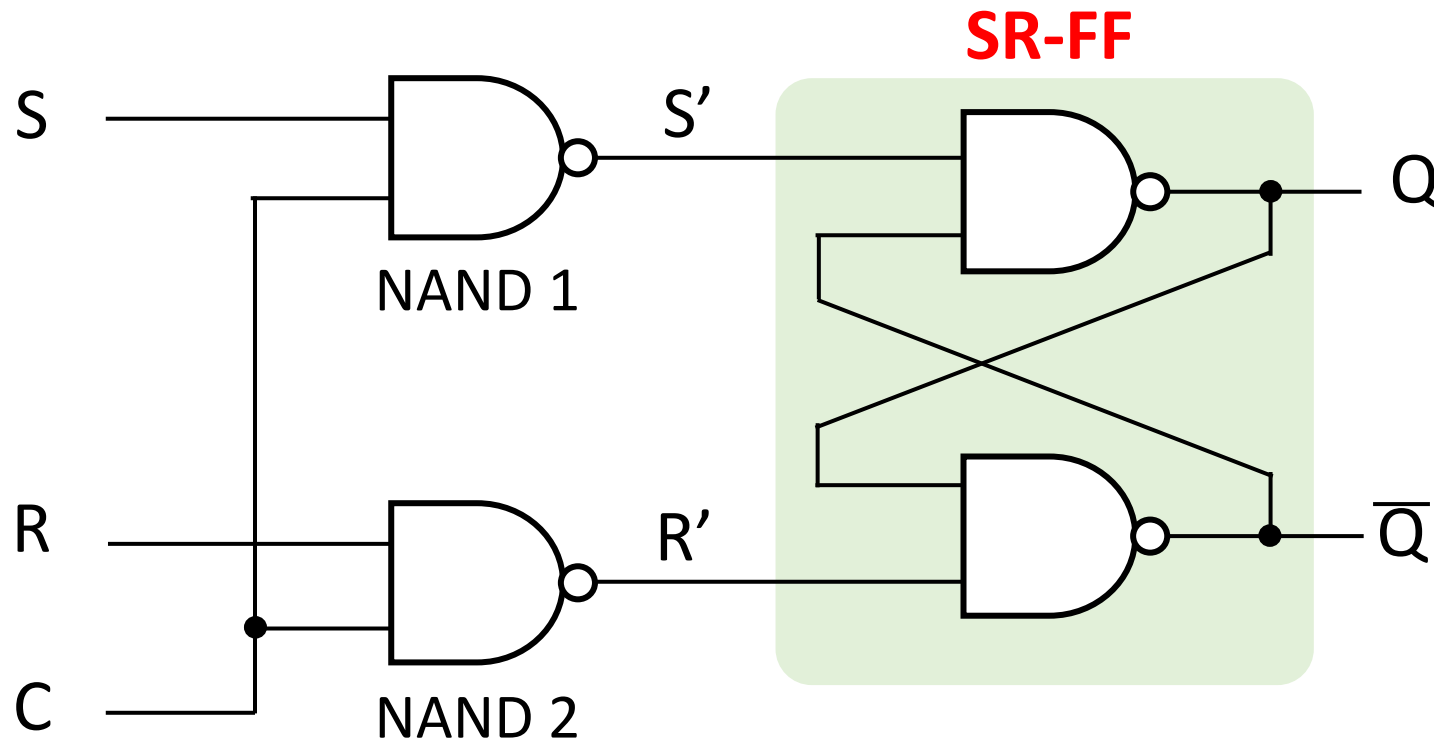
- クロックに合わせたSet/Reset動作を行う場合のタイミングチャート
 - クロックが上がっているときに, Set($Q=1$)やReset($Q=0$)にする
 - クロックが下がっているときは, 何もしない



3-3. 同期式SR-フリップフロップの回路

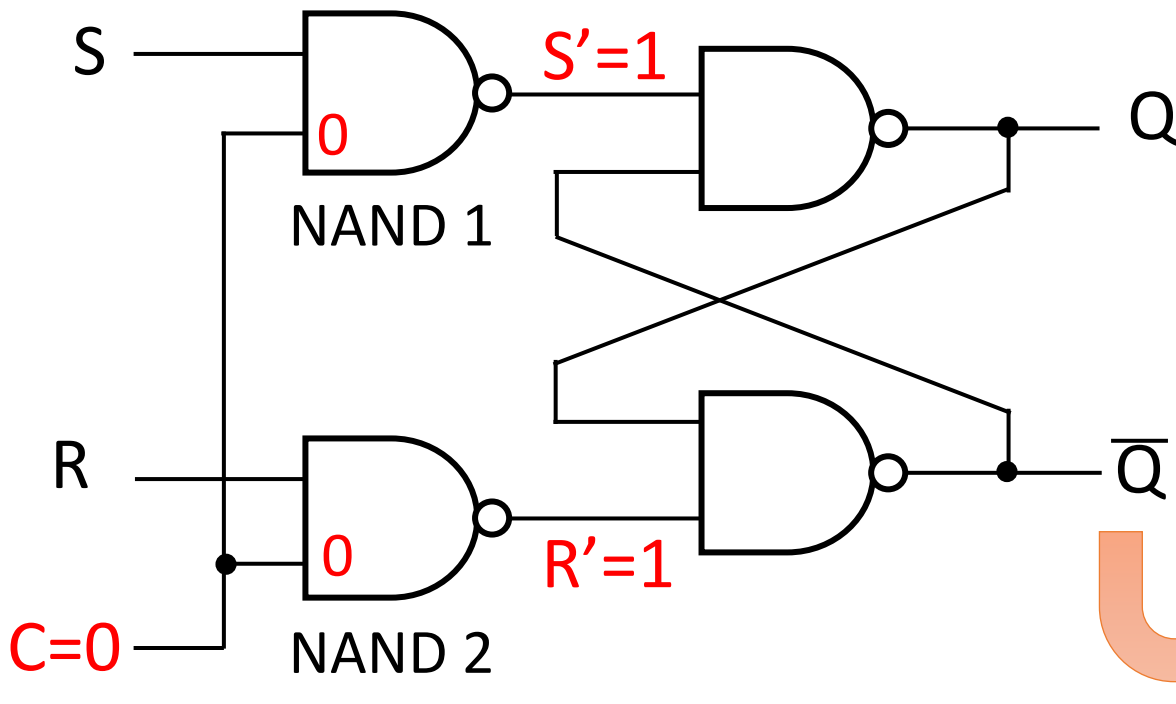
- 論理回路図

- 通常SR-FFの前段に、NANDゲートを2つ配置している
- 入力は、S(セット)、R(リセット)、C(クロック)の3つ
- 入力Sと入力Rに $\sim$ (バー)が無いので、**正論理**になっている



3-4. 同期式SR-フリップフロップの動作(1)

- 3-3の回路図を基に動作を確認。**クロックC=0**の場合
 - NAND1とNAND2がともに1を出力。つまり、 S' と R' が1になる
 - SR-FFの $\sim S$ と $\sim R$ がともに1の場合、「保持」になり、 **S や R が何であろうと Q は変化なし**



NAND真理値表

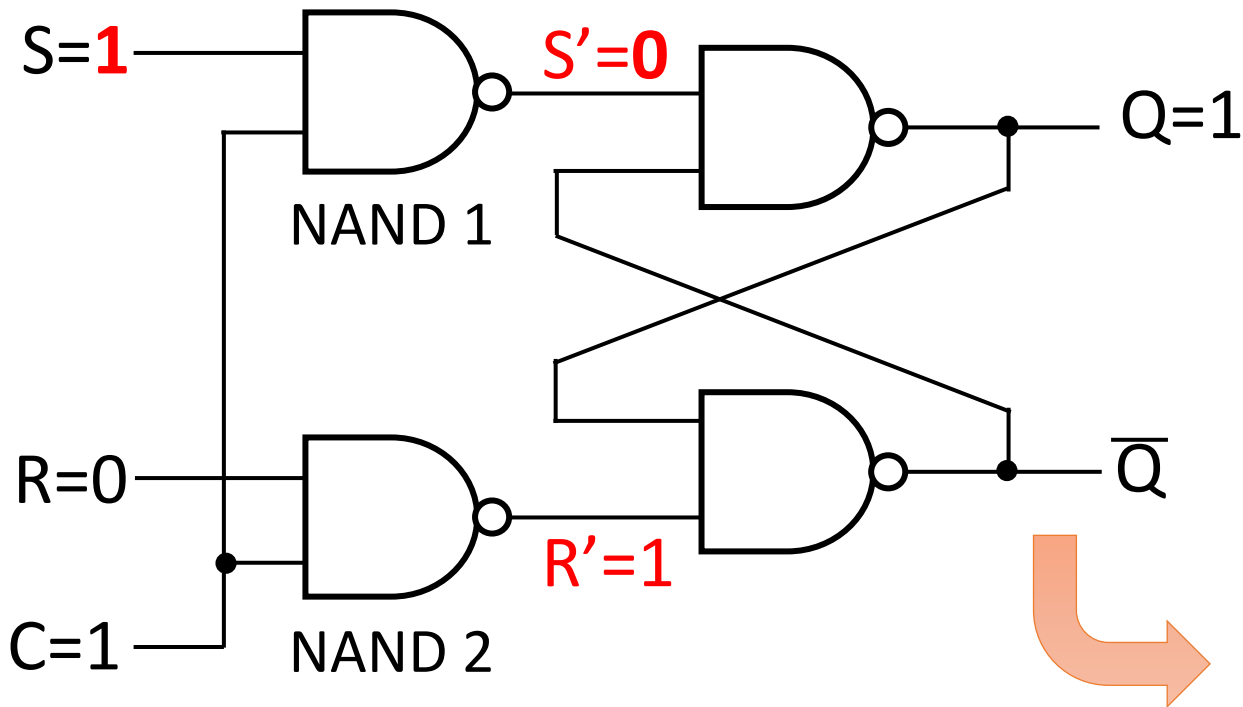
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

S	R	C	S'	R'	Q
0/1	0/1	0	1	1	保持

3-4. 同期式SR-フリップフロップの動作(2)

- 3-3の回路図を基に動作を確認。クロックC=1, S=1, R=0の場合
 - NAND1とNAND2がインバータになる
 - S=1だから $S'=0$, R=0だから $R'=1$ になる。さらに, SR-FFの動作から, QがSetされる



NAND真理値表

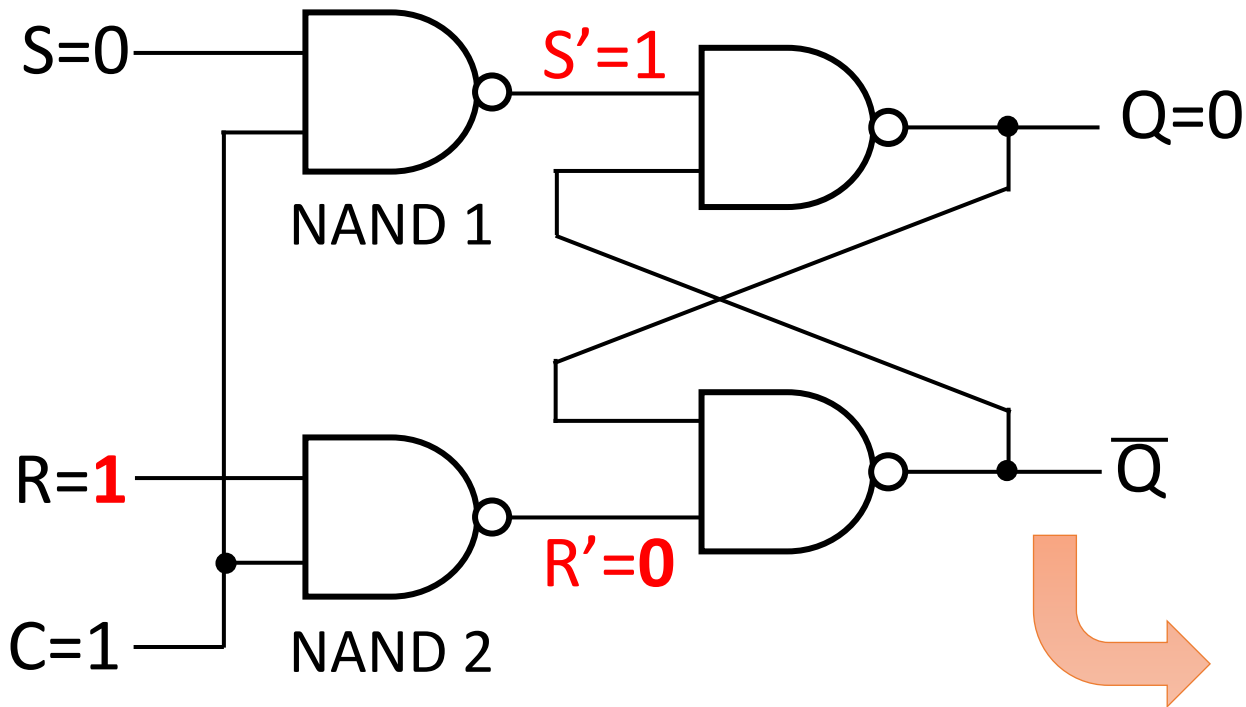
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

S	R	C	S'	R'	Q
1	0	1	0	1	1

3-4. 同期式SR-フリップフロップの動作(3)

- 3-3の回路図を基に動作を確認。クロックC=1, S=0, R=1の場合
 - NAND1とNAND2がインバータになる
 - S=0だから $S'=1$, $R=1$ だから $R'=0$ になる。さらに, SR-FFの動作から, QがResetされる



NAND真理値表

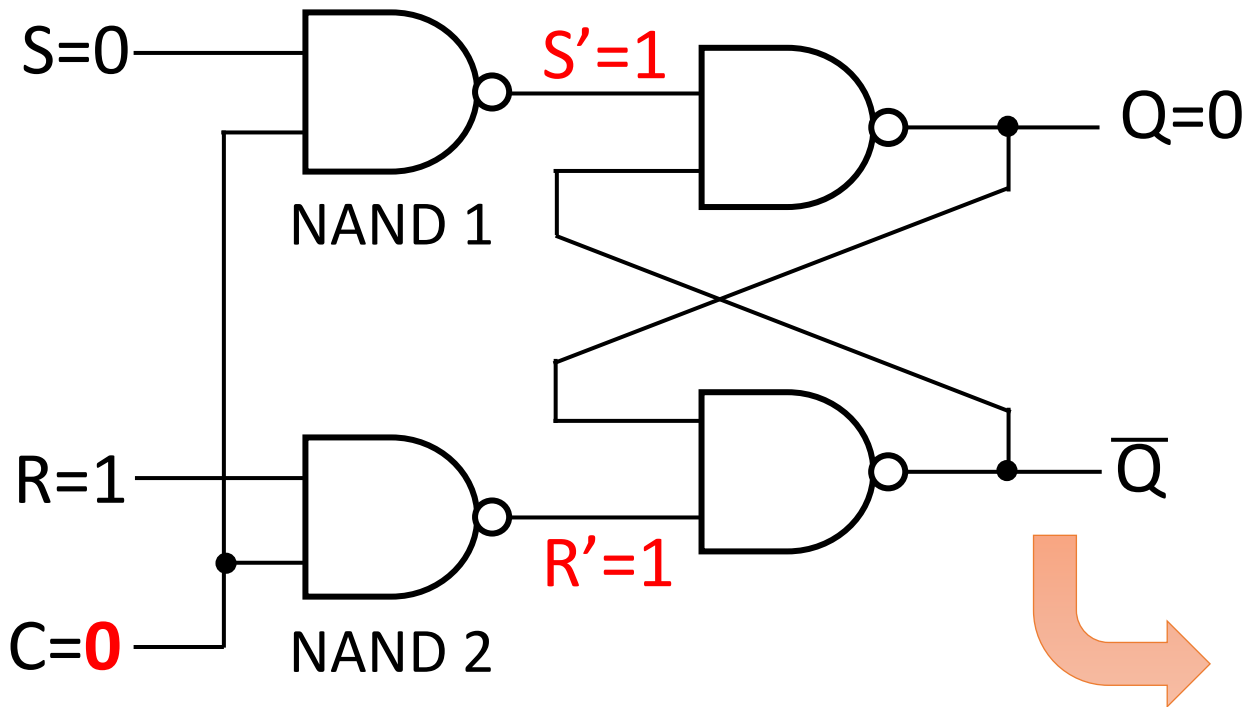
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

S	R	C	S'	R'	Q
0	1	1	0	1	0

3-4. 同期式SR-フリップフロップの動作(4)

- 3-3の回路図を基に動作を確認。**クロックC=0**に戻った場合
 - NAND1とNAND2がともに1を出力。つまり、 S' と R' が1になる
 - SR-FFの $\sim S$ と $\sim R$ がともに1の場合、「保持」になる**



NAND真理値表

A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

S	R	C	S'	R'	Q
0	1	0	1	1	保持

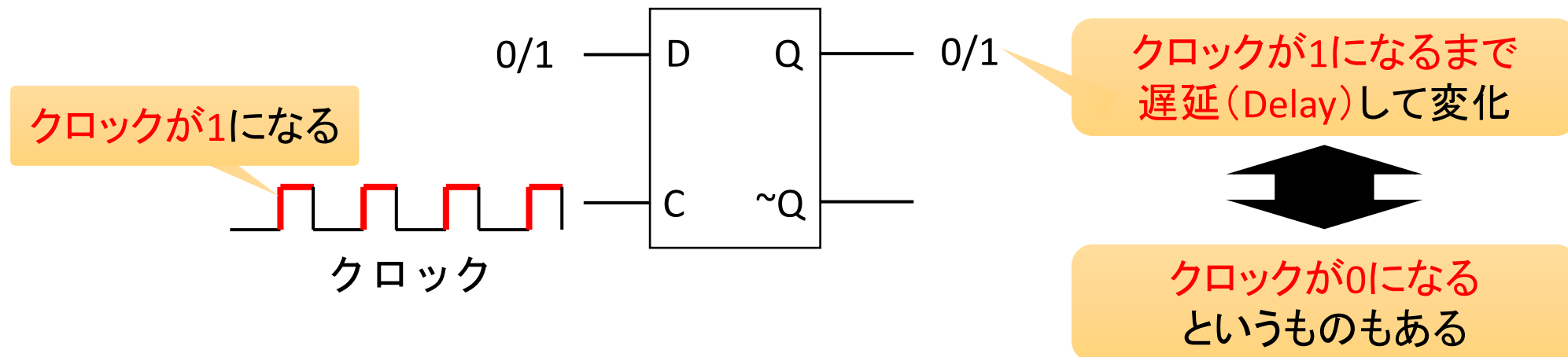
3-5. 同期式SR-フリップフロップまとめ

- SR-FFの前段にNANDゲートが2つ接続された構成
- SR-FFの前段のNANDゲートが, SR-FFへの入力を調整
- 入力
 - セットS, リセットR, クロックCの3入力。SR-FFと違い, **正論理**になる点に注意
- 動作
 - クロックCが1の間だけ, SetとResetの設定が有効になる
 - クロックCが0になると, 保持状態になり, SetとResetは効かない

4. Dラッチ(レベル・センスD-FF)

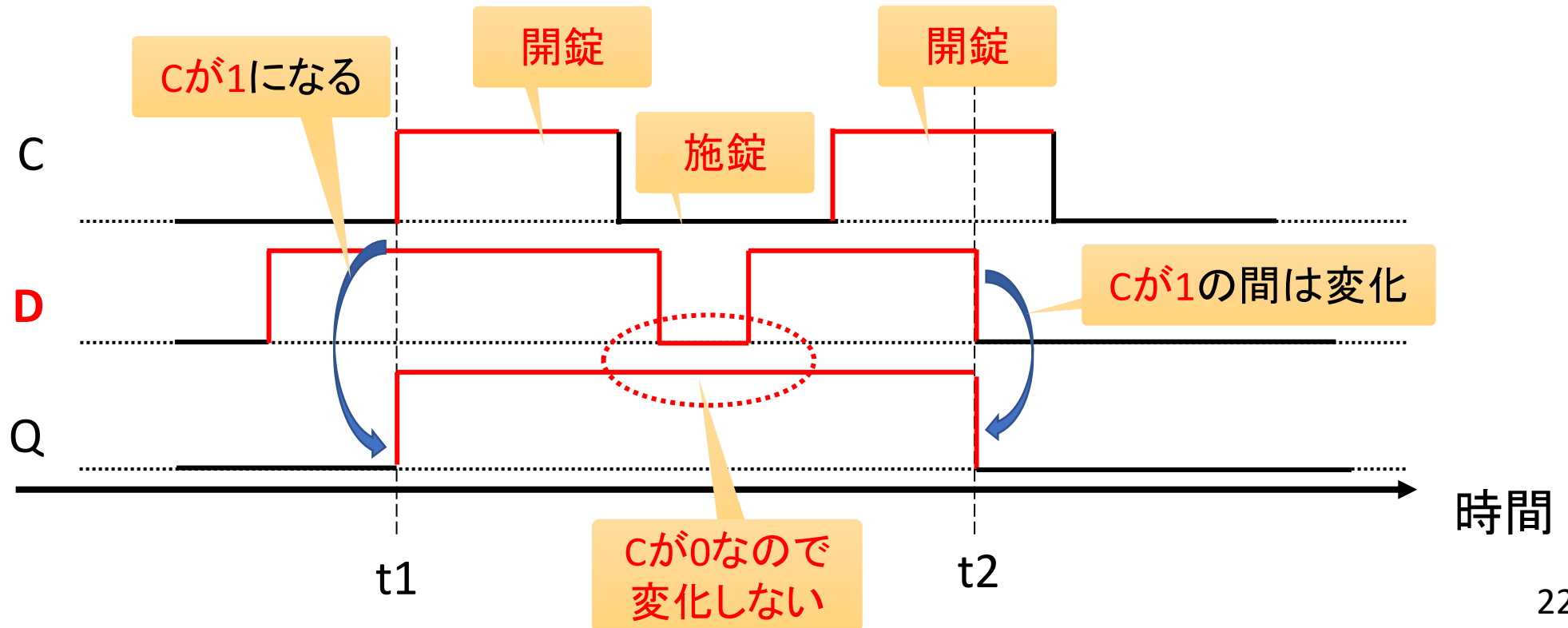
4-1. D-フリップフロップとは

- 「D-フリップフロップ」の「D」は, Delay (遅延) のこと。クロックが1になるまで, 出力Qが変化することを遅延
 - D-フリップフロップは, クロックに合わせて, 出力Qが0や1になる
 - つまり, クロックが1の時に, 出力Qを0や1にできる
- D-フリップフロップには, 入力がDしかない。Dを1にすると出力Qが1に, Dを0にすると出力Qが0になる
- クロックの扱い方によって, D-ラッチやエッジトリガ式D-FFなどに分かれる



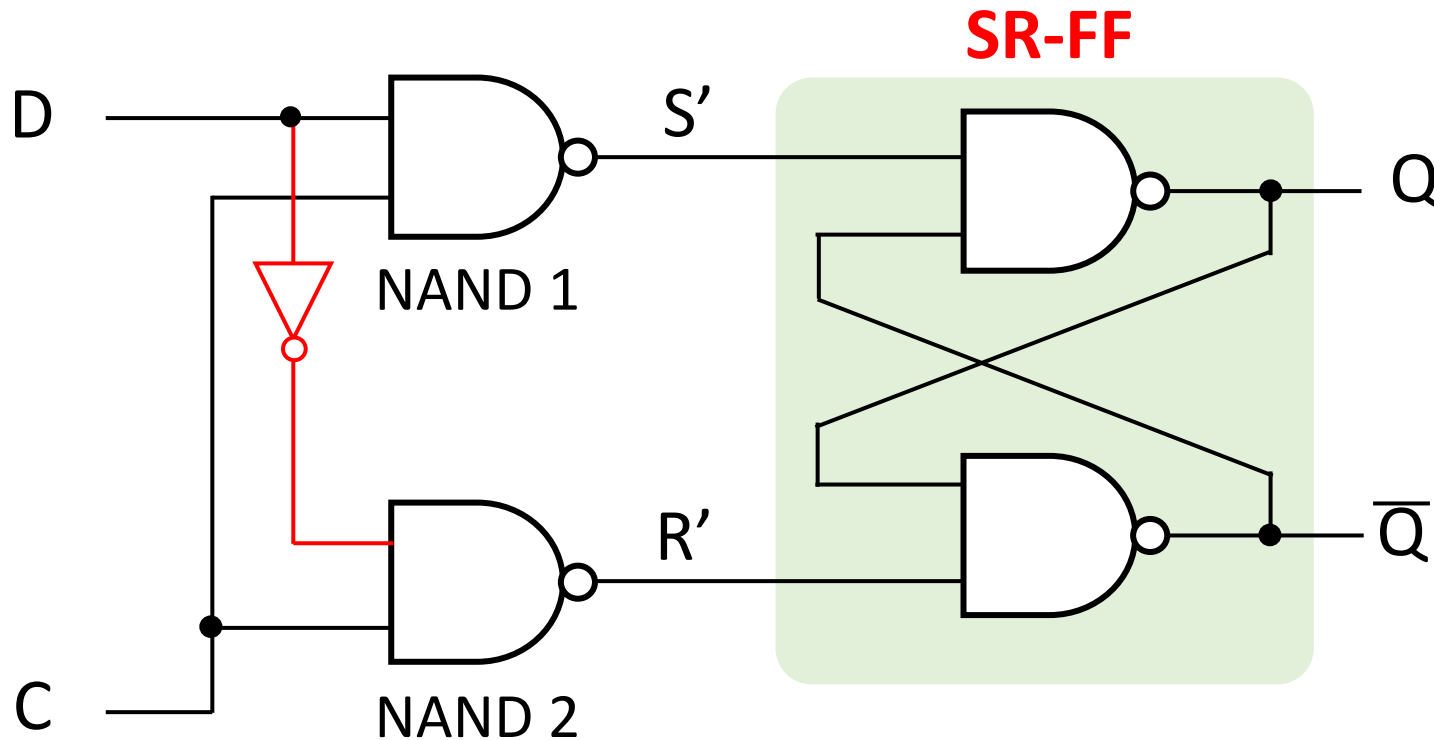
4-2. D-ラッチとタイミングチャート

- クロックCが1の間は，入力Dの変化が出力Qに伝わるD-FF
 - ラッチとは，扉の止め金具のこと
 - C=1なら扉が開いて，C=0なら閉じるというイメージから，D-ラッチと呼ぶ(らしい)
- D-ラッチのタイミングチャート



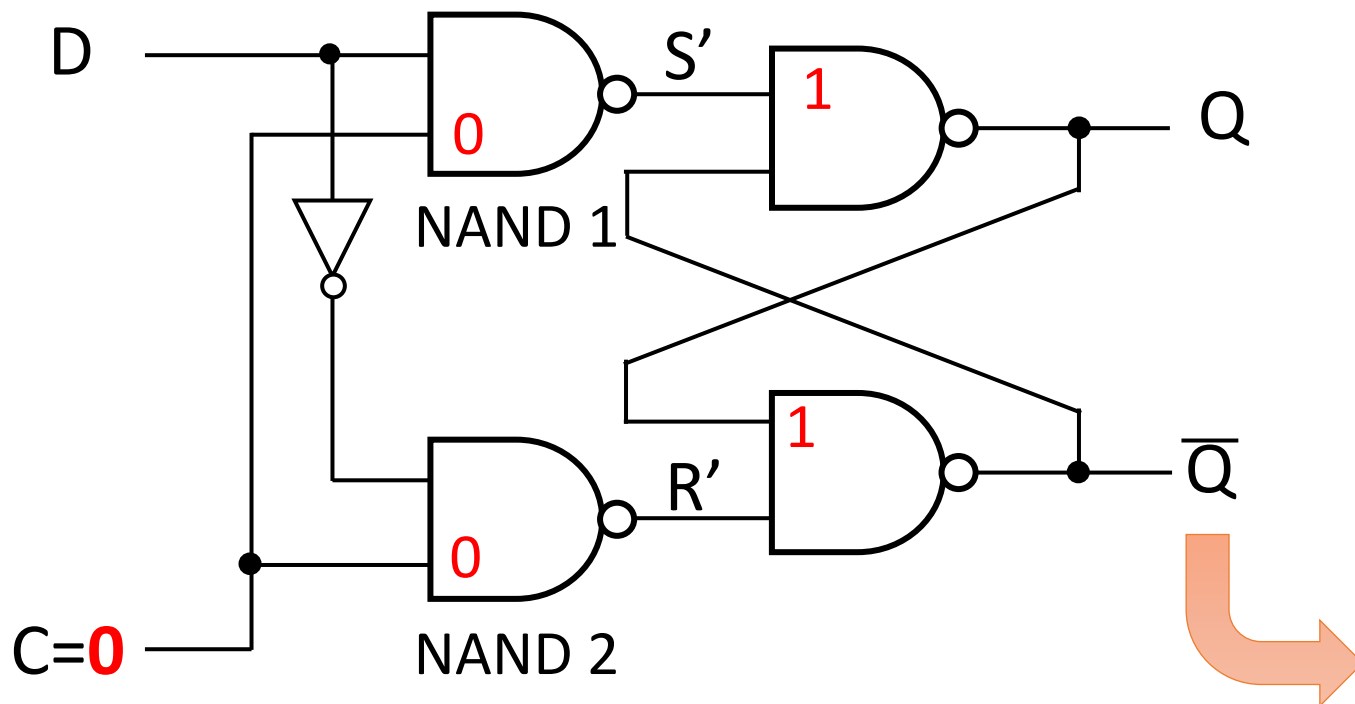
4-3. D-ラッチの回路

- 同期式SR-FFの入力Sが入力Dになり，入力Rが $\sim D$ になる。ただし， $\sim D$ は，わざわざ入れなくてよい
 - SR-FFでSetを上げると，自動的にResetが落ちる仕掛け。逆にSetを落とすと，Resetが上がる。合理的ですね



4-4. D-ラッチの動作(1)

- 4-3の回路図を基に動作を確認。**クロックC=0**の場合
 - NAND1とNAND2がともに1を出力。つまり、 S' と R' が1になる
 - SR-FFの $\sim S$ と $\sim R$ がともに1の場合、「保持」になり、**Qは変化**なし(同期式SR-FFと同じ)



NAND真理値表

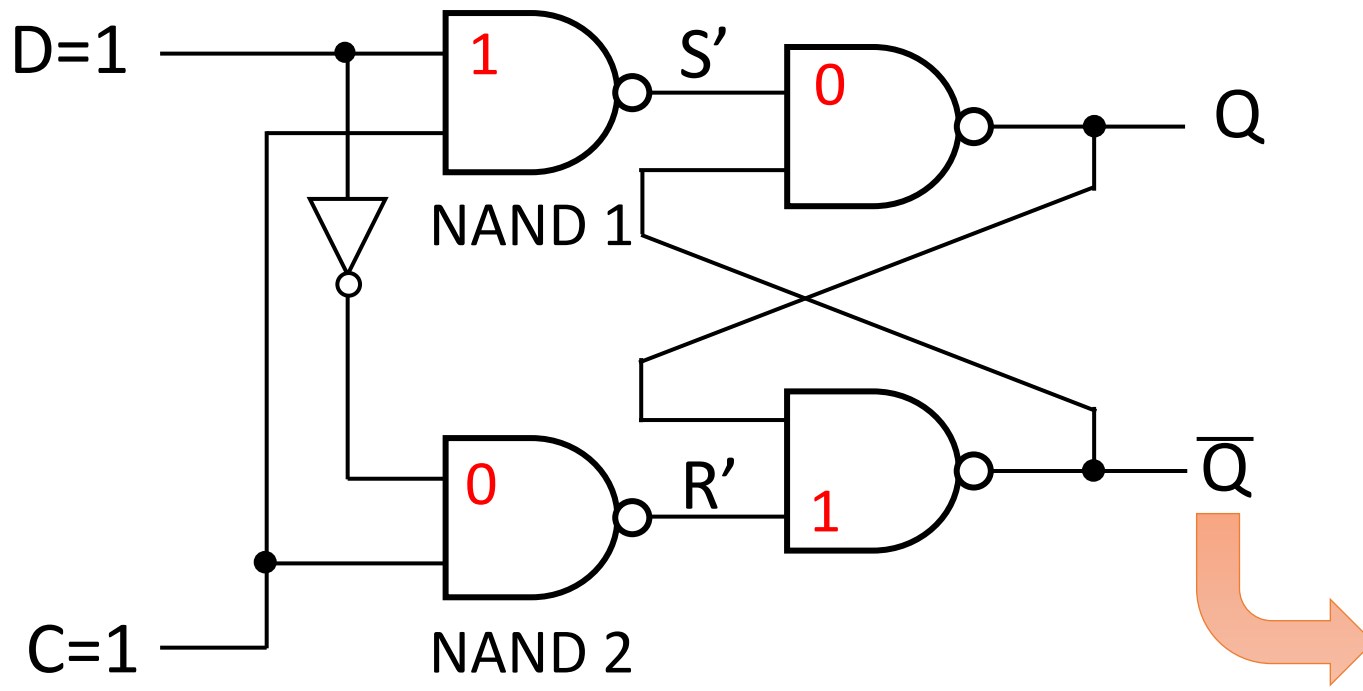
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

D	C	S'	R'	Q
0/1	0	1	1	保持

4-4. D-ラッチの動作(2)

- クロックC=1, D=1の場合
 - NAND1が0, NAND2が1を出力。つまり, $S'=0$ と $R'=1$ になる
 - SR-FFは負論理だから, $Q=1$ になる($\sim Q=0$)



NAND真理値表

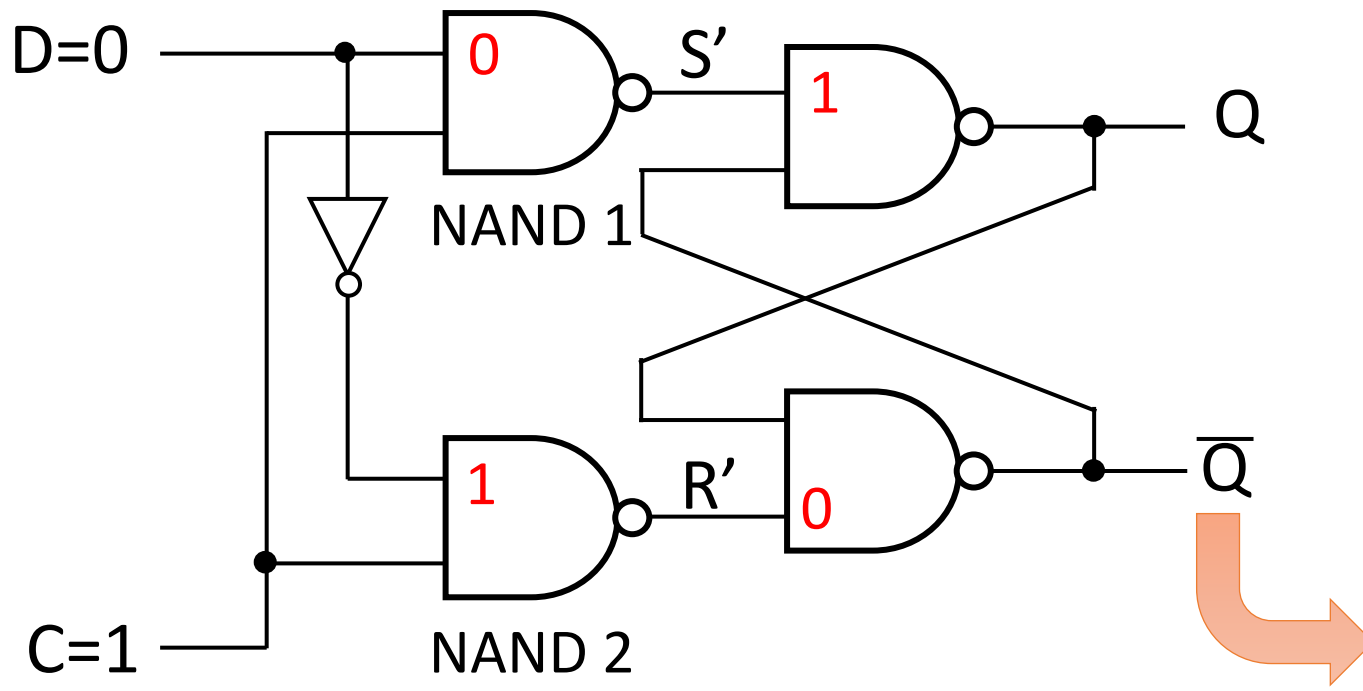
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

D	C	S'	R'	Q
1	1	0	1	1

4-4. D-ラッチの動作(3)

- クロックC=1, D=0の場合
 - NAND1が0, NAND2が1を出力。つまり, $S'=1$ と $R'=0$ になる
 - SR-FFは負論理だから, $Q=0$ になる($\sim Q=1$)



NAND真理値表

A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

クロック

D	C	S'	R'	Q
0	1	1	0	0

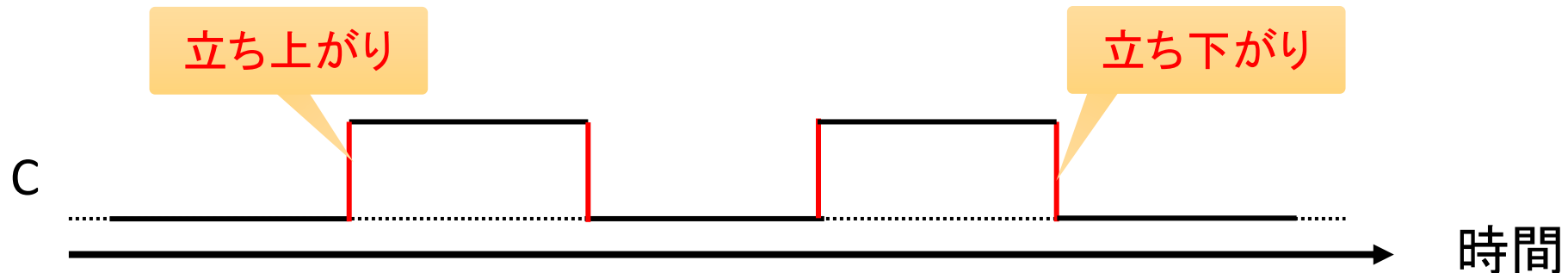
4-5. D-ラッチのまとめ

- SR-FFの前段にNANDゲートが2つ接続された構成。SR-FFと比べて、入力数が1つ少ない
- クロックCが扉の鍵のような役割になる。クロックCが1の時だけ入力Dの設定が出力Qに伝わる。クロックCが0の時は、変化なし
- 入力
 - 入力DとクロックCの2入力。正論理になる点に注意
- 動作
 - クロックCが1の間だけ、入力Dの設定が有効になる
 - クロックCが0になると、保持状態になり、入力Dは効かない

5. エッジトリガ式D-FF

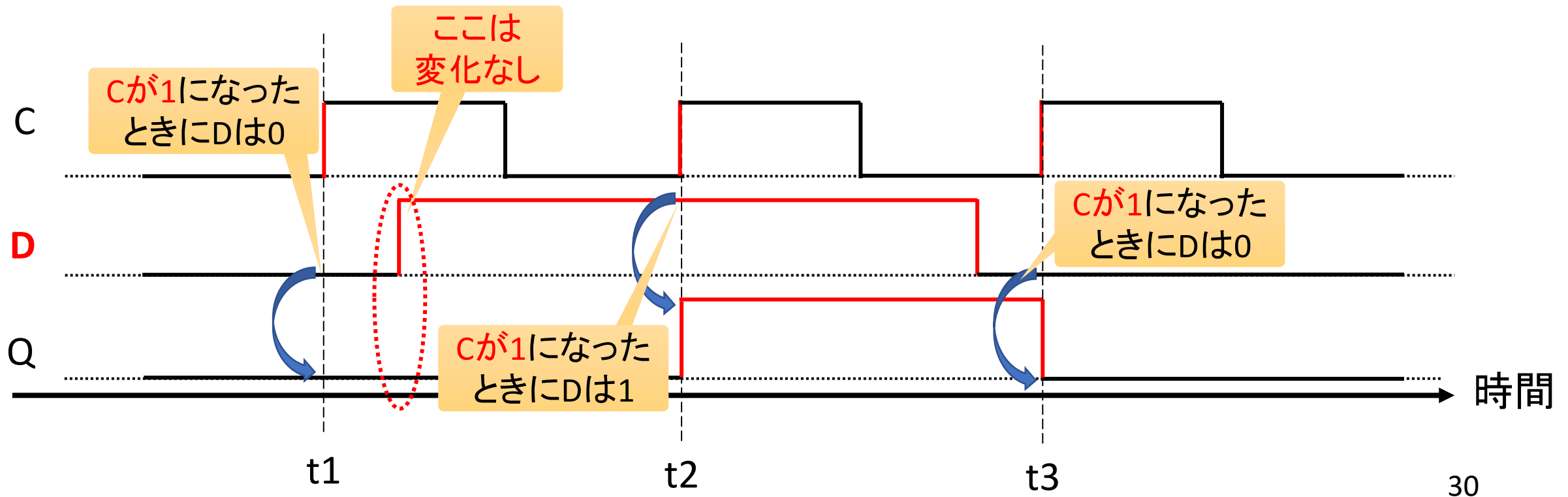
5-1. エッジトリガとは

- クロックの変化のタイミングで状態が変化するエッジトリガ
 - エッジ(Edge)は, 縁, 境界などの意味を持つ英語
 - トリガ(Trigger)は, 引き金という意味を持つ英語
 - 「エッジトリガ」は縁で引き金を引くということ
- 「エッジトリガ式」は, クロックの立ち上がりや立ち下がり, 出力Qが変化する。それ以外は変化しない



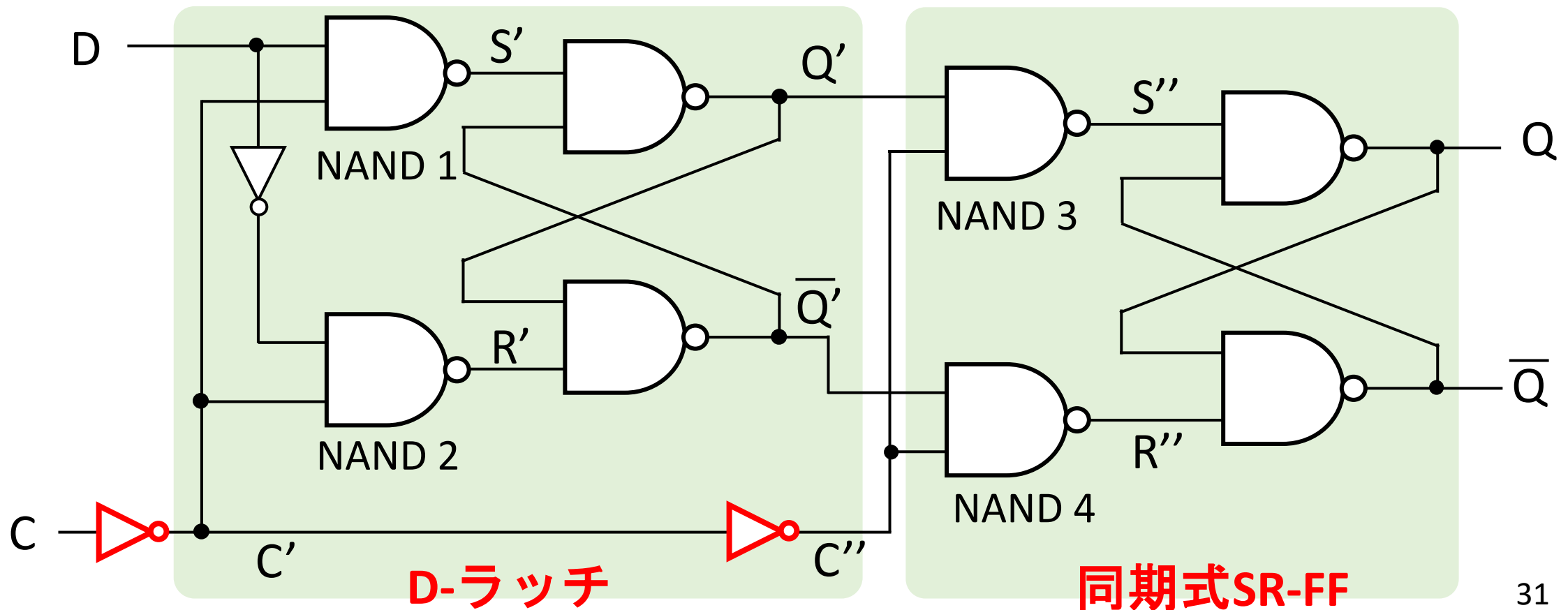
5-2. エッジトリガ式D-FFとタイミングチャート

- クロックCが立ち上がる時に、入力Dの変化が出力Qに伝わるD-FF
 - 立ち下がりのエッジトリガ式では**ない**D-FF
 - D-ラッチとは異なり、クロックCが1の間にDが変化しても、Qに影響しない
- エッジトリガ式D-FFのタイミングチャート



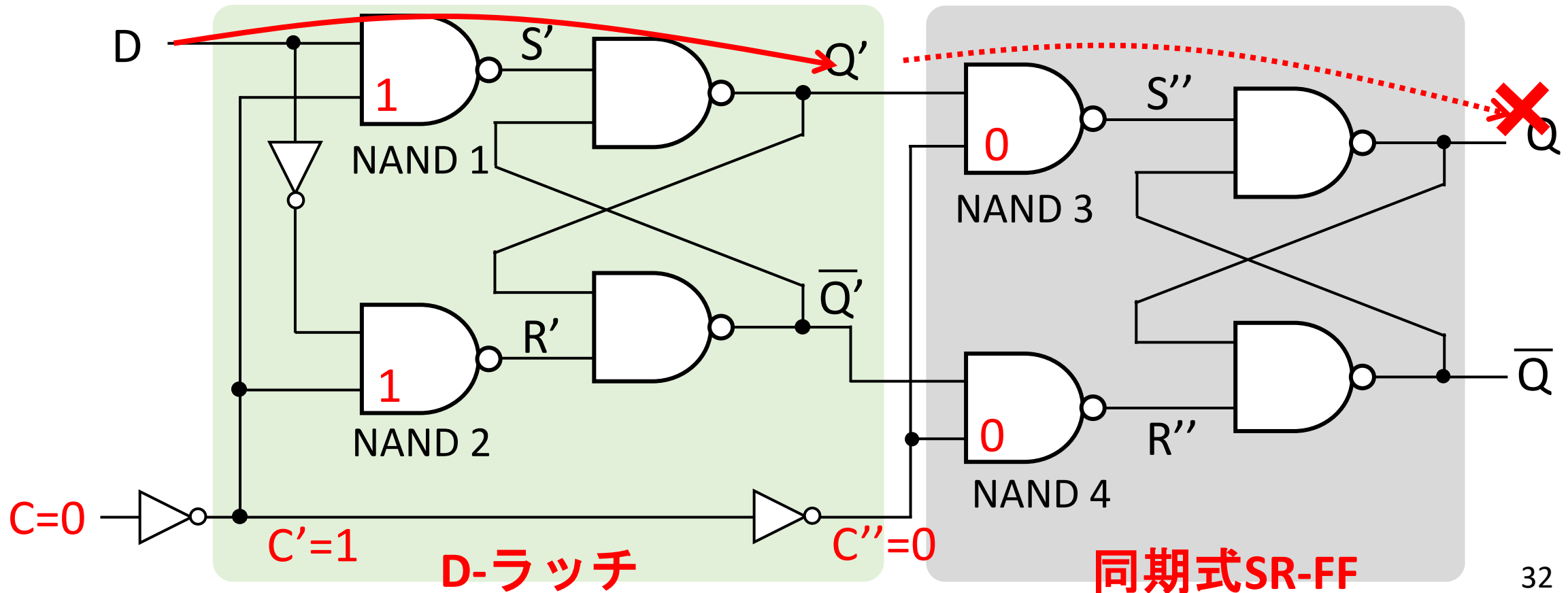
5-3. エッジトリガ式D-FFの回路

- D-ラッチと同期式SR-フリップフロップを組み合わせた回路
 - 左側がD-ラッチで、右側がSR-FFになっている
 - クロックCはインバータで反転されている。ここで、一方のFFのみ有効にする



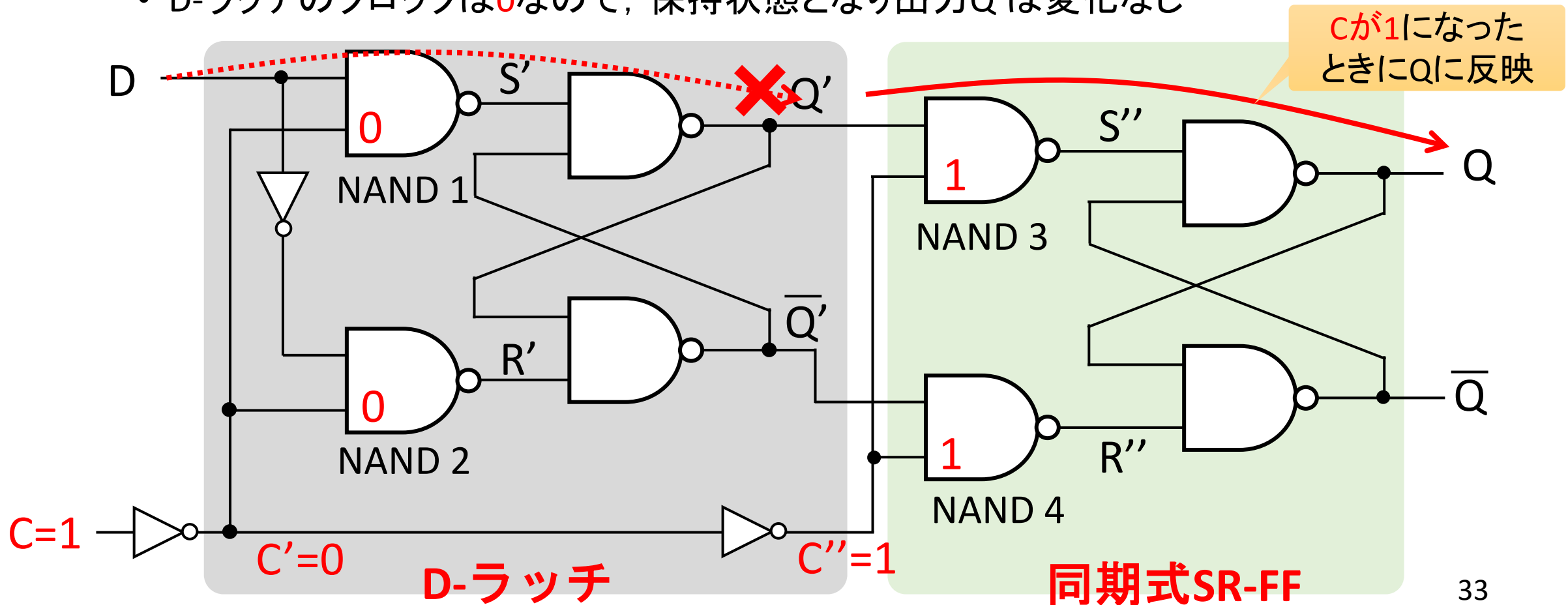
5-4. エッジトリガ式D-FFの動作(1)

- 5-3の回路図を基に動作を確認。**クロックC=0**の場合
 - C'が1になり、D-ラッチが有効になる。一方、C''は0になり同期式SR-FFは無効
 - 入力Dの値がD-ラッチの出力Q'と $\bar{Q}'$ に反映される
 - ただし、同期式SR-FFはクロックが0なので、保持状態となっており、出力Qは変化なし



5-4. エッジトリガ式D-FFの動作(2)

- 5-3の回路図を基に動作を確認。**クロックC=1**の場合
 - C'が0になり、D-ラッチが**無効**。一方、C''は0になり同期式SR-FFは**有効**
 - D-ラッチの出力Q'が、同期式SR-FFの入力となり、Qに反映される
 - D-ラッチのクロックは**0**なので、保持状態となり出力Q'は変化なし



5-5. エッジトリガ式D-FFのまとめ

- D-ラッチと同期式SR-FFを組み合わせた構成。クロックをずらして入力する
- クロックが0の時, Dラッチが有効, 同期式SR-FFが無効
- クロックが1の時, Dラッチが無効, 同期式SR-FF有効
- 入力
 - 入力DとクロックCの2入力。正論理になる点に注意
- 動作
 - クロックCが1になったときに, QにDが反映
 - クロックCに0や1が継続している間は, Qは変化なし

演習

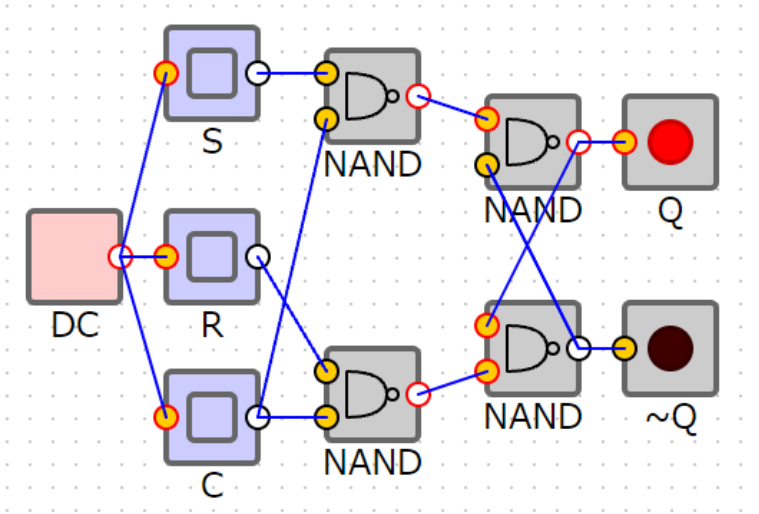
これまでと同様に、シミュレータで作成した論理回路をMoodleに提出してください
(11:45～12:15)

演習問題

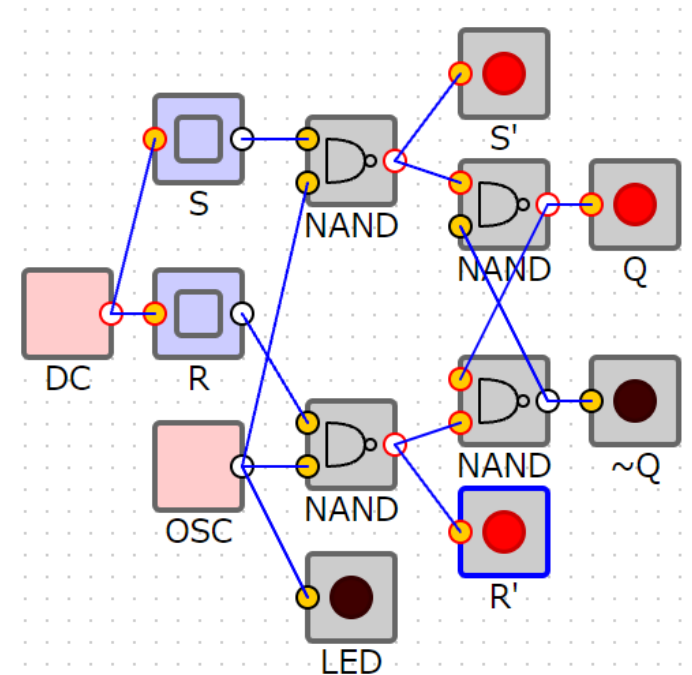
- 今回登場したフリップフロップ回路を論理回路シミュレータで作成
 1. 同期式SR-FF
 2. Dラッチ
 3. エッジトリガ式D-FF

同期式SR-フリップフロップ

- 論理回路シミュレータで作成
 - クロックCの動作は, 自分で入力
 - OSC(オシレータ)を使って, クロックを入れても良いが, 操作が難しい...



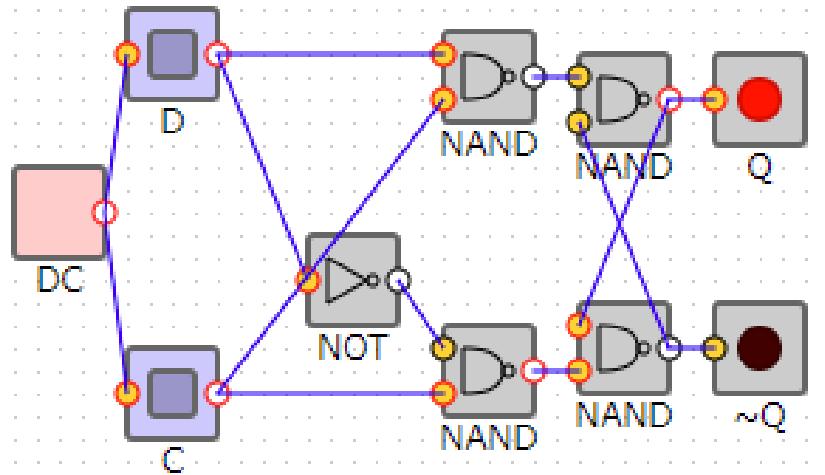
(1) 手動クロック



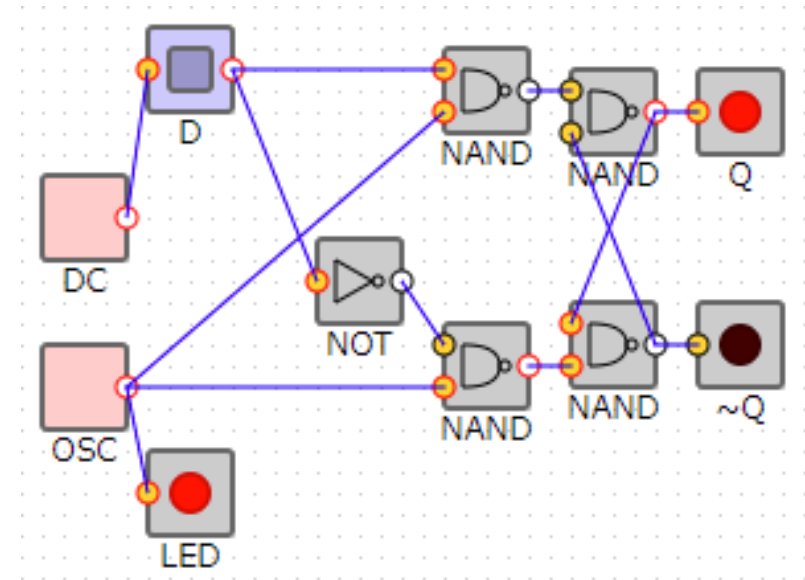
(2) OSCクロック

D-ラッチ

- 論理回路シミュレータで作成
 - クロックCの動作は, 自分で入力
 - OSC(オシレータ)を使って, クロックを入れても良いが, 操作が難しい...



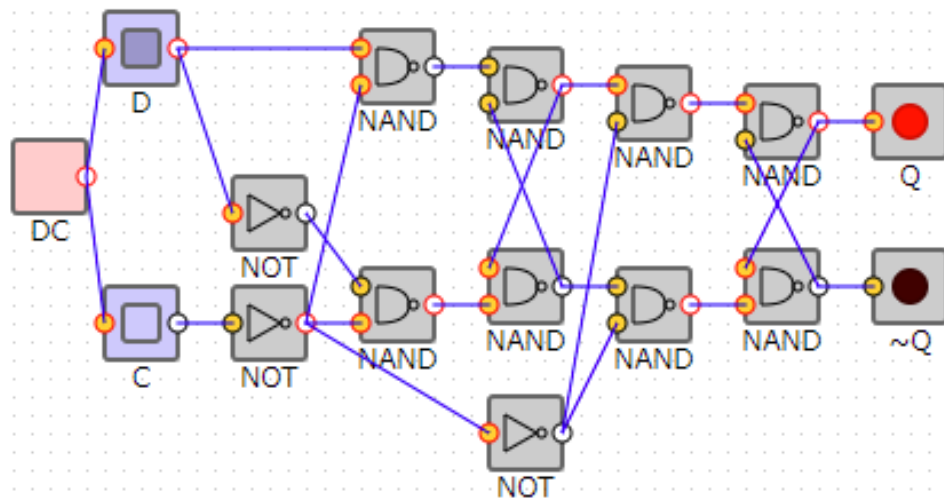
(1) 手動クロック



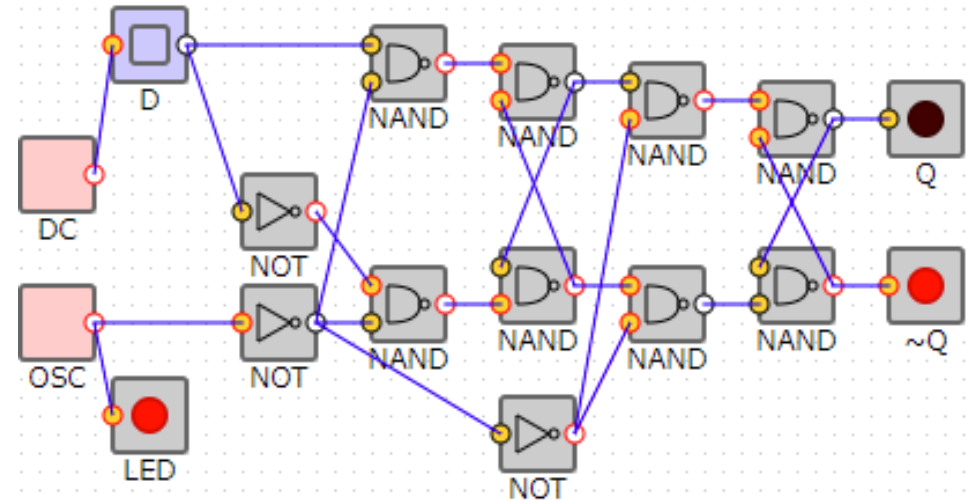
(2) OSCクロック

エッジトリガ式D-FF

- 論理回路シミュレータで作成
 - クロックCの動作は, 自分で入力
 - OSC(オシレータ)を使って, クロックを入れても良いが, 操作が難しい...



(1) 手動クロック



(2) OSCクロック

まとめ

- 以下を解説しました
 - SR-フリップフロップに無かった「クロック」の概念を理解
 - クロックに基づいて動作する同期式SR-フリップフロップの動作の理解
 - Dラッチ(Dフリップフロップ)とエッジトリガ式Dフリップフロップの動作の理解
 - SR-フリップフロップ, Dラッチ, エッジトリガ式Dフリップフロップのタイミングチャート
 - 同期式SR-フリップフロップとDラッチを論理回路シミュレータで動作させて実践的に理解
- 論理回路をシミュレータで動作確認して理解を深めてください